Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут ім. Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЛАБОРАТОРНА РОБОТА № 3

з курсу «Організація баз даних»

Виконали:
Студенти ІІ курсу ФІОТ
Групи ІО-45
Бондарчук М.О.
Романенко І.Є.

Перевірив:
Русінов Володимир Володимирович

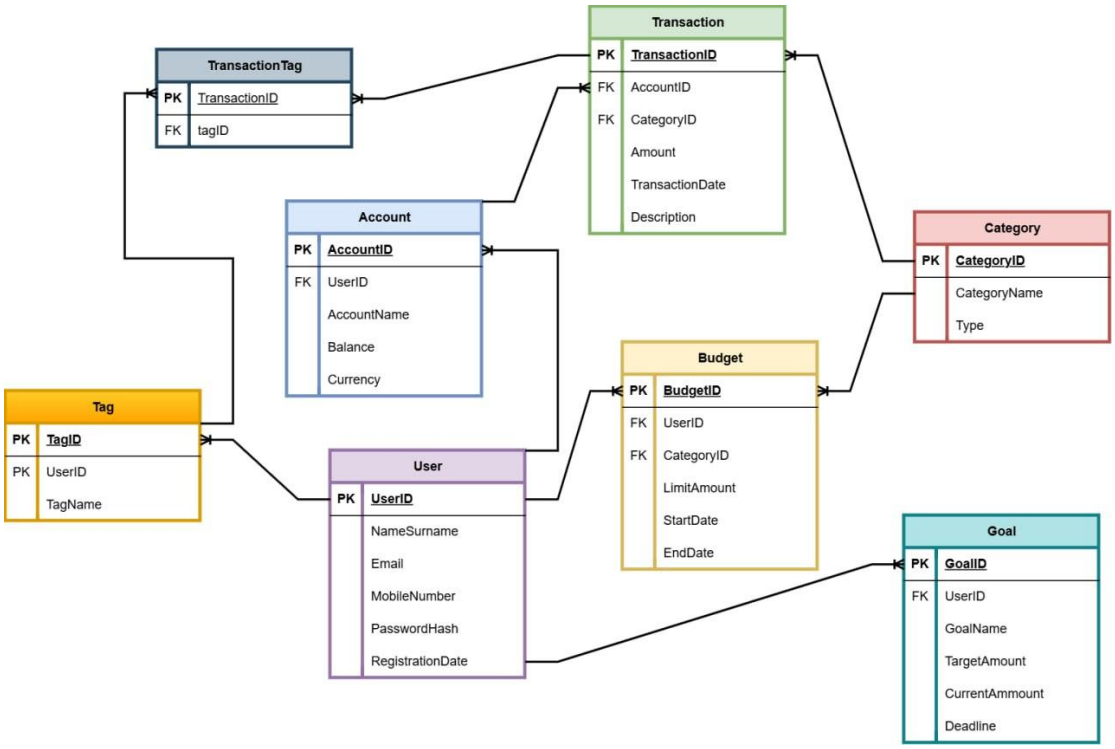
Київ 2026

Мета лабораторної роботи : протестувати базу даних, яку створили в лабораторних роботах 1–2, виконуючи запити в стилі транзакцій у PostgreSQL (за допомогою pgAdmin). Ми зосереджуємося на запитах OLTP (онлайн-обробка транзакцій) – великій кількості коротких транзакцій, які отримують або оновлюють окремі записи. На практиці це означає написання ручних SQL-інструкцій для вибору даних та вставки, оновлення або видалення рядків. Це імітує реальні та перевіряє, чи ваша схема та дані поводяться належним чином.

Цілі

- Написати запити **SELECT** для отримання даних (включаючи фільтрацію за допомогою WHERE та вибір певних стовпців).
- Практикувати використання оператори **INSERT** для додавання нових рядків до таблиць.
- Практикувати використання оператора **UPDATE** для зміни існуючих рядків (використовуючи SET та WHERE).
- Практикувати використання оператори **DELETE** для безпечного видалення рядків (за допомогою WHERE).
- Вивчити основні операції маніпулювання даними (DML) у PostgreSQL та спостерігайте за їхнім впливом.

ER-діаграма:



1. Оператор SELECT (Отримання даних)

Мета: витягнути дані з таблиць, використовуючи фільтрацію WHERE та вибір конкретних стовпців. SELECT лише отримує дані і нічого не змінює.

```
-- 1.1. Вибрати лише імена та пошти всіх користувачів (без паролів та іншого)
SELECT name_surname, email
FROM users;

-- 1.2. Вибрати всі транзакції, сума яких перевищує 500 грн (використання WHERE)
SELECT transaction_id, amount, description, transaction_date
FROM transactions
WHERE amount > 500.00;

-- 1.3. Знайти конкретний фінансовий рахунок за його назвою
SELECT account_id, balance, currency
FROM accounts
WHERE account_name = 'Монобанк';
```

	account_id [PK] integer	balance numeric (12,2)	currency character varying (10)
1	1	15000.00	UAH

2. Оператор INSERT (Додавання даних)

Мета: попрактикувати додавання нових рядків до таблиць. Для кожної таблиці потрібно вказати стовпці та відповідні значення.

```
-- 2.1. Додаємо нову категорію витрат
INSERT INTO categories (category_name, type)
VALUES ('Медицина', 'Expense');

-- 2.2. Додаємо тестову транзакцію для користувача (наприклад, купівля ліків з рахунку 1)
-- Важливо: переконайтеся, що категорія 'Медицина' отримала ID (наприклад, 4).
-- Якщо її ID інший, замініть цифру 4 у запиті нижче.
INSERT INTO transactions (account_id, category_id, amount, description)
VALUES (1, 4, 1500.00, 'Аптека: вітаміни та ліки');
```

Data Output Messages Notifications

Showing rows: 1

	transaction_id [PK] integer	account_id integer	category_id integer	amount numeric (12,2)	transaction_date timestamp without time zone	description text
1	1	1	1	25000.00	2026-03-07 10:24:35.878221	Аванс
2	2	1	3	350.00	2026-03-07 10:24:35.878221	Сніданок в Пузатій Х...
3	3	1	3	120.00	2026-03-07 10:24:35.878221	Кава біля метро
4	4	1	4	1500.00	2026-03-15 15:34:04.265392	Аптека: вітаміни та лі...

3. Оператор UPDATE (Оновлення даних)

Мета: змінити існуючі рядки за допомогою SET та WHERE.

Критичне правило: Завжди використовувати WHERE, інакше буде зміна даних абсолютно у всіх рядках таблиці.

```
-- 3.1. Змінюємо баланс рахунку (наприклад, після витрати грошей баланс Монобанку зменшився)
UPDATE accounts
SET balance = 13500.00
WHERE account_id = 1;

-- 3.2. Виправляємо опечатку в описі конкретної транзакції (припустимо, її ID = 4)
UPDATE transactions
SET description = 'Аптека: ліки від застуди'
WHERE transaction_id = 4;
```

Showing rows: 1

	transaction_id [PK] integer	account_id integer	category_id integer	amount numeric (12,2)	transaction_date timestamp without time zone	description text
1	1	1	1	25000.00	2026-03-07 10:24:35.878221	Аванс
2	2	1	3	350.00	2026-03-07 10:24:35.878221	Сніданок в Пузатій Хаті
3	3	1	3	120.00	2026-03-07 10:24:35.878221	Кава біля метро
4	5	1	4	1500.00	2026-03-15 15:35:37.164351	Аптека: вітаміни та лі...
5	4	1	4	1500.00	2026-03-15 15:34:04.265392	Аптека: ліки від засту...

4. Оператор DELETE (Видалення даних)

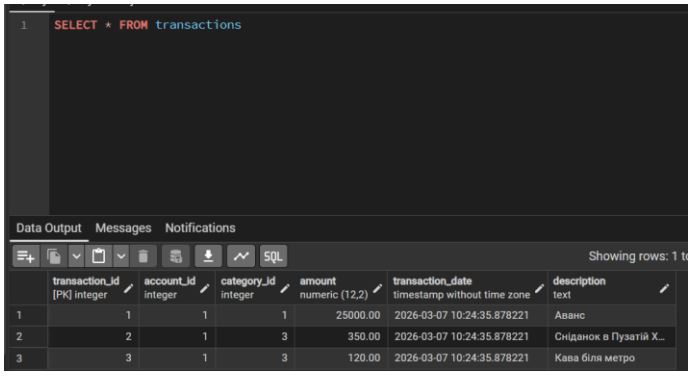
Мета: безпечно видалити рядки за допомогою WHERE. Якщо не включити WHERE, уся таблиця буде очищена.

Будемо видаляти тестову категорію або транзакцію, яку створили.

Примітка: через зовнішні ключі (Foreign Keys) не можна видалити категорію, якщо до неї вже прив'язані транзакції. Тому спочатку видаляємо транзакцію.

```
-- 4.1. Спочатку видаляємо транзакцію (або транзакції), пов'язану з категорією "Медицина"
DELETE FROM transactions
WHERE category_id = 4;

-- 4.2. Тепер, коли категорія вільна від зв'язків, база дозволить її безпечно видалити
DELETE FROM categories
WHERE category_id = 4;
```



```
SELECT * FROM transactions
```

Data Output Messages Notifications

Showing rows: 1 t

	transaction_id [PK] integer	account_id integer	category_id integer	amount numeric (12,2)	transaction_date timestamp without time zone	description text
1	1	1	1	25000.00	2026-03-07 10:24:35.878221	Аванс
2	2	1	3	350.00	2026-03-07 10:24:35.878221	Сніданок в Пузатій Х...
3	3	1	3	120.00	2026-03-07 10:24:35.878221	Кава біля метро

У ході виконання третьої лабораторної роботи ми успішно закріпили навички роботи з мовою маніпулювання даними (DML) у СУБД PostgreSQL. На базі розробленої нами схеми бази даних для вебзастосунку ведення особистого бюджету ми протестували базові OLTP-операції:

1. Навчилися витягувати необхідну інформацію за допомогою запитів SELECT із застосуванням фільтрації WHERE.
2. Відпрацювали додавання нових транзакцій та категорій за допомогою INSERT INTO.
3. Успішно оновили існуючі дані (баланс рахунків та описи) за допомогою безпечних команд UPDATE із вказуванням умов.
4. Протестували видалення записів за допомогою команди DELETE.

Особливо цінним практичним досвідом стало зіткнення з помилкою порушення цілісності зовнішнього ключа (Foreign Key constraint violation) під час спроби видалити категорію, до якої були прив'язані транзакції. Це наочно продемонструвало, як реляційна база даних самостійно захищає дані від появи «осиротілих» записів і зберігає логіку нашого фінансового застосунку. Виконані запити повністю імітують реальні транзакції, які буде виконувати бекенд нашого проєкту.